

Jobsheet 6 — Fetch API & JSON

Sub-CPMK: Menerapkan komunikasi asinkron (AJAX/fetch, JSON).

Perubahan dari Jobsheet 5

- Tambah `data/buku.json` (10 objek) dan `data/anggota.json` (4 objek) sebagai pengganti sementara API sungguhan.
- `buku/list.html` & `anggota/list.html`: `<tbody>` dikosongkan, baris kini dirender dinamis oleh `assets/js/buku.js` / `assets/js/anggota.js` menggunakan `fetch + async/await`.
- Loading indicator (`#loading-indicator`) tampil selama proses fetch (disimulasikan dengan delay 600ms).
- Penanganan error (`try/catch`) menampilkan pesan di dalam tabel bila fetch gagal.
- `app.js`: `initHapusConfirm` diubah ke **event delegation** (`document.addEventListener("click", ...)`) karena tombol Hapus sekarang berada di baris yang dibuat setelah halaman selesai dimuat.

Cara menjalankan

Penting: `fetch()` ke file lokal akan diblokir kebijakan CORS jika dibuka langsung dengan `file://`. Jalankan lewat server lokal, misalnya:

```
php -S localhost:8000
```

lalu buka `http://localhost:8000/index.html`. Bisa juga memakai ekstensi "Live Server" di VSCode.

Catatan

- Uji error handling dengan mengganti sementara nama file di `fetch(...)` menjadi nama yang salah.
 - Pola `fetch + async/await` di sini akan dipakai ulang untuk memanggil endpoint PHP sungguhan mulai Jobsheet 9 (setelah back-end PostgreSQL siap di Jobsheet 8), meskipun mulai Jobsheet 7 rendering utama berpindah ke server-side PHP.
-

Dokumentasi Jobsheet 6 – Fetch API & JSON

Dokumentasi ini melanjutkan [dokumentasi jobsheet-05](#) (JavaScript DOM & Event). Kalau kamu belum paham dasar JavaScript – memilih elemen DOM, event listener, `classList`, dsb. – baca dulu dokumentasi jobsheet-05 sebelum lanjut ke sini, karena jobsheet ini banyak membangun di atas konsep tersebut.

Tentang docs/wireframe.md

File ini **identik persis** dengan [docs/wireframe.md](#) di [jobsheet-05](#) – tidak ada rancangan UI/UX baru di jobsheet ini. Baca [dokumentasi jobsheet-04](#) kalau perlu menyegarkan ingatan soal wireframe & user flow.

Apa yang Baru di Jobsheet 6?

Sesuai [README.md](#) jobsheet ini, ini titik penting lain dalam perjalanan SIMPUS-Mini: untuk **pertama kalinya**, data tabel (Daftar Buku & Daftar Anggota) **tidak lagi ditulis manual** di HTML – diambil secara dinamis dari file JSON memakai JavaScript. Empat perubahan besarnya:

1. **Dua file JSON baru** (`data/buku.json`, `data/anggota.json`) – sumber data, menggantikan sementara API/server sungguhan yang belum ada.
2. **Rendering tabel dipindah ke JavaScript** – `<tbody>` di HTML sekarang **kosong**, diisi dinamis oleh `assets/js/buku.js` / `assets/js/anggota.js` lewat `fetch` + `async/await`.
3. **Loading indicator** – teks "Memuat data..." muncul sesaat selagi data sedang diambil.
4. **Penanganan error** dengan `try/catch` – kalau pengambilan data gagal, tabel menampilkan pesan error alih-alih halaman kosong/rusak.

Ditambah satu perubahan pendukung: `initHapusConfirm` di `app.js` diubah ke pola **event delegation**, karena tombol Hapus sekarang berada di baris yang baru dibuat setelah halaman selesai dimuat (dibahas di [bab 6](#)).

Daftar Isi

1. [Konsep Dasar: AJAX, JSON, Promise, async/await](#)
2. [Apa yang Berubah di File HTML?](#)
3. [Data JSON: `buku.json` & `anggota.json`](#)
4. [JS: Mengambil & Menampilkan Daftar Buku](#)
5. [JS: Mengambil & Menampilkan Daftar Anggota](#)

6. [JS: Event Delegation pada Tombol Hapus](#)
7. [Menjalankan Lewat Server Lokal \(CORS\)](#)
8. [Rangkuman & Latihan Lanjutan](#)

Struktur Folder

```
jobsheet-06/
├── index.html
├── assets/
│   ├── css/style.css      # Tidak berubah dari jobsheet-05
│   └── js/
│       ├── app.js         # initHapusConfirm diubah (event delegation)
│       ├── buku.js        # BARU - fetch + render Daftar Buku
│       └── anggota.js     # BARU - fetch + render Daftar Anggota
├── data/
│   ├── buku.json          # BARU - 10 objek data buku
│   └── anggota.json       # BARU - 4 objek data anggota
├── buku/
│   ├── list.html         # <tbody> kosong + loading-indicator
│   └── tambah.html
├── anggota/
│   ├── list.html
│   └── tambah.html
├── docs/wireframe.md      # Identik dengan jobsheet-05
├── README.md
└── Dokumentasi/          # Folder dokumentasi ini
```

Catatan penting sejak awal (dari [README.md](#) jobsheet ini): `data/buku.json` dan `data/anggota.json` adalah **pengganti** sementara** untuk API sungguhan. Pola `fetch + async/await` yang kamu** pelajari di sini akan dipakai ulang untuk memanggil endpoint PHP asli mulai Jobsheet 9 — meskipun mulai Jobsheet 7, rendering utama justru berpindah ke sisi server (PHP), bukan lagi sepenuhnya di browser seperti di jobsheet ini.

1. Konsep Dasar: AJAX, JSON, Promise, async/await

Sub-CPMK jobsheet ini menyebut "komunikasi asinkron (AJAX/fetch, JSON)" — empat istilah yang perlu dipahami dulu sebelum membaca kode `buku.js` / `anggota.js`.

1.1 Apa itu AJAX?

AJAX (*Asynchronous JavaScript and XML* — namanya historis, sekarang hampir selalu dipakai dengan JSON, bukan XML) adalah teknik mengambil data dari server **tanpa perlu me-reload seluruh halaman**. Bandingkan dengan form yang sudah kamu kenal sejak [dokumentasi jobsheet-01](#): men-submit form biasa membuat browser memuat ulang **seluruh** halaman. Dengan AJAX, JavaScript bisa mengambil data baru di **latar belakang**, lalu memperbarui **sebagian** halaman saja (di jobsheet ini: mengisi `<tbody>` tabel) — pengalaman yang jauh lebih mulus bagi pengguna.

1.2 Apa itu JSON?

JSON (*JavaScript Object Notation*) adalah format teks untuk menyimpan data terstruktur, dirancang supaya mudah dibaca manusia **dan** mudah diproses program. Lihat `data/buku.json`:

```
[
  { "judul": "Laskar Pelangi", "pengarang": "Andrea Hirata", "tahun": 2005, "stok": 4 },
  { "judul": "Bumi Manusia", "pengarang": "Pramoedya Ananta Toer", "tahun": 1980, "stok": 2 }
]
```

- Kurung siku `[ ]` di luar menandakan sebuah **array** (daftar/list) berisi banyak item.
- Setiap item dibungkus kurung kurawal `{ }` — ini disebut **objek**, kumpulan pasangan **kunci** (*key*) dan **nilai** (*value*), dipisah titik dua. Contoh: kunci `"judul"` bernilai `"Laskar Pelangi"`.
- Bandingkan strukturnya dengan satu baris `<tr>` di tabel HTML statis yang sudah kamu kenal sejak [dokumentasi jobsheet-01](#): satu objek JSON `{ "judul": ..., "pengarang": ..., "tahun": ..., "stok": ... }` membawa informasi yang **persis sama** dengan satu baris `<tr>` yang berisi 4 sel `<td>` — hanya bentuk penyimpanannya berbeda. Penjelasan detail kedua file JSON ini ada di [bab 3](#).

Penting: JSON punya aturan penulisan yang lebih ketat daripada objek JavaScript biasa — nama kunci **wajib** diapit tanda kutip dua (`"judul"`, bukan `judul` tanpa kutip), dan JSON **tidak mengizinkan** komentar sama sekali. Aturan ketat ini sengaja dibuat supaya format JSON bisa diproses secara konsisten oleh hampir semua bahasa pemrograman, bukan cuma JavaScript.

1.3 Apa itu `fetch()`?

`fetch(url)` adalah fungsi bawaan browser untuk **meminta** data dari sebuah alamat (bisa file lokal seperti `data/buku.json`, atau alamat server sungguhan). Bentuk paling sederhananya:

```
fetch("../data/buku.json")
```

Baris ini saja **belum** memberi kita data — ia hanya **memulai** permintaan. Untuk memahami kenapa dibutuhkan langkah tambahan, kita perlu paham dulu konsep **Promise**.

1.4 Apa itu Promise? (Kenapa Butuh `await`?)

Mengambil data lewat jaringan **butuh waktu** — bisa beberapa milidetik, bisa juga beberapa detik tergantung kecepatan koneksi. JavaScript **tidak berhenti total** menunggu proses ini selesai (kalau berhenti total, seluruh halaman akan "membeku" selama menunggu) — sebagai gantinya, `fetch()` langsung mengembalikan sebuah **Promise**: sebuah "janji" bahwa hasilnya (data sungguhan, atau error) akan tersedia **nanti**, bukan seketika itu juga.

`await` adalah kata kunci yang berarti "tunggu sampai Promise ini selesai, baru lanjutkan ke baris berikutnya" — tanpa `await`, kita hanya akan mendapat "janji"-nya, bukan data sungguhan:

```
const res = await fetch("../data/buku.json");
```

Baris ini berarti: "mulai ambil data dari `buku.json`, **tunggu** sampai prosesnya selesai, baru simpan hasilnya ke `res`."

1.5 Apa itu `async function`?

`await` **hanya boleh dipakai** di dalam fungsi yang ditandai `async` di depannya:

```
async function muatDaftarBuku() {  
  // boleh pakai "await" di sini  
}
```

Kata kunci `async` memberi tahu JavaScript bahwa fungsi ini **berjalan** secara asinkron** — ia boleh "berhenti sejenak" di baris yang ada** `await`-nya (menunggu Promise selesai) tanpa memblokir seluruh halaman untuk melakukan hal lain sementara itu. Pasangan `async / await` ini adalah cara **modern** menulis kode asinkron di JavaScript — jauh lebih mudah dibaca dibanding cara-cara lama (`.then()` / `.catch()` berantai) yang tidak dipakai di jobsheet ini.

1.6 Menangani Kegagalan: `try / catch / finally`

```
try {
  // kode yang mungkin gagal (misalnya fetch gagal karena file tidak ada)
} catch (err) {
  // dijalankan HANYA kalau ada error di blok try
} finally {
  // selalu dijalankan, entah berhasil atau gagal
}
```

- `try { ... }` membungkus kode yang berpotensi gagal.
- `catch (err) { ... }` menangkap error itu (disimpan ke variabel `err`) supaya program **tidak berhenti total/crash** — alih-alih, kita bisa menampilkan pesan yang ramah ke pengguna (lihat penerapannya di [bab 4](#)).
- `finally { ... }` selalu dijalankan di akhir, **baik** `try` berhasil **maupun** `catch` menangani error — cocok untuk kode yang harus selalu dijalankan apa pun hasilnya, seperti menyembunyikan loading indicator (dijelaskan di [bab 4](#)).

Dengan bekal 6 konsep di atas (AJAX, JSON, `fetch`, `Promise/await`, `async function`, `try/catch/finally`), kamu siap membaca kode `buku.js` dan `anggota.js` baris demi baris mulai bab 4.

Lanjut ke: [Apa yang Berubah di File HTML?](#)

2. Apa yang Berubah di File HTML?

2.1 `<tbody>` yang Sekarang Kosong

Perbandingan langsung dengan `buku/list.html` di [dokumentasi jobsheet-01](#): dulu `<tbody>` berisi 5 baris `<tr>` yang ditulis manual. Sekarang:

```
<tbody>
  <!-- Baris diisi dinamis oleh assets/js/buku.js via fetch('../data/buku.json') -->
</tbody>
```

`<tbody>` **benar-benar kosong** — hanya berisi komentar HTML (`<!-- ... -->`, tidak tampil di layar, sekadar catatan untuk siapa pun yang membaca kode) yang menjelaskan bahwa baris-barisnya akan diisi belakangan oleh JavaScript setelah halaman dimuat. Kalau kamu membuka file ini langsung tanpa JavaScript aktif, tabelnya akan **tampil kosong** — hanya baris judul kolom (`<thead>`) yang terlihat.

2.2 Elemen Baru: Loading Indicator

```
<p id="loading-indicator" style="display:none;">Memuat data...</p>
```

Elemen `<p>` baru diletakkan di antara kolom pencarian ([dokumentasi jobsheet-05 §2.2](#)) dan tabel. Dua hal yang perlu diperhatikan:

- `id="loading-indicator"` — "kait" yang dicari JavaScript lewat `document.getElementById("loading-indicator")` (dijelaskan di [bab 4](#)).
- `style="display:none;"` — ini contoh **inline style**, CSS yang ditulis **langsung** di atribut `style` sebuah elemen HTML, bukan di file `style.css` terpisah seperti yang selalu kamu lakukan sejak [dokumentasi jobsheet-02](#). Elemen ini **sengaja disembunyikan sejak awal** (`display:none`) karena teks "Memuat data..." hanya perlu terlihat **sesaat**, selama proses pengambilan data berlangsung — JavaScript yang akan menampilkan dan menyembunyikannya kembali secara terprogram (dijelaskan di [bab 4 §4.3](#)). Karena gaya ini hanya dipakai **satu kali** di satu elemen spesifik dan statusnya akan terus diubah lewat JavaScript (bukan lewat selector CSS), menulisnya inline di sini lebih praktis dibanding membuat aturan baru khusus di `style.css`.

2.3 Urutan Tag `<script>` yang Baru

```
<script src="../assets/js/app.js"></script>
<script src="../assets/js/buku.js"></script>
```

Sekarang ada **dua** tag `<script>` di `buku/list.html` (dan `anggota/list.html` memuat `app.js` + `anggota.js`), dimuat **berurutan** dari atas ke bawah — sama seperti browser membaca HTML dari atas ke bawah yang sudah dibahas di [dokumentasi jobsheet-05 §1.3](#). `app.js` (berisi fungsi-fungsi umum seperti menu hamburger, validasi form, filter tabel — lihat [dokumentasi jobsheet-05](#)) selalu dimuat **lebih dulu**, baru disusul file khusus halaman (`buku.js` atau `anggota.js`) yang isinya spesifik untuk satu halaman itu saja. Halaman `buku/tambah.html` dan Beranda **tidak** memuat `buku.js` / `anggota.js` sama sekali — karena halaman itu tidak punya tabel yang perlu diisi data dinamis.

2.4 Kenapa `buku.js` dan `anggota.js` Dipisah, Bukan Digabung ke `app.js`?

Ini keputusan desain yang baik untuk dipahami: `app.js` berisi fungsi-fungsi yang **relevan di banyak halaman sekaligus** (menu hamburger muncul di semua halaman, validasi form hanya di

halaman "tambah", filter tabel di halaman "list"). Sedangkan logika mengambil data JSON **spesifik** untuk satu jenis data (buku **atau** anggota) – memisahkannya ke file sendiri-sendiri membuat tiap file lebih pendek, lebih mudah dicari, dan halaman yang tidak membutuhkannya (misalnya Beranda) tidak perlu memuat kode yang tidak relevan sama sekali.

Lanjut ke: [Data JSON: buku.json & anggota.json](#)

3. Data JSON: buku.json & anggota.json

Sebelum membedah kode yang **membaca** file JSON ini, kenali dulu isi dan strukturnya.

3.1 data/buku.json — 10 Objek Buku

```
[
  { "judul": "Laskar Pelangi", "pengarang": "Andrea Hirata", "tahun": 2005, "stok": 4 },
  { "judul": "Bumi Manusia", "pengarang": "Pramoedya Ananta Toer", "tahun": 1980, "stok": 2 },
  { "judul": "Negeri 5 Menara", "pengarang": "Ahmad Fuadi", "tahun": 2009, "stok": 0 },
  { "judul": "Filosofi Teras", "pengarang": "Henry Manampiring", "tahun": 2018, "stok": 5 },
  { "judul": "Ronggeng Dukuh Paruk", "pengarang": "Ahmad Tohari", "tahun": 1982, "stok": 1 },
  { "judul": "Cantik Itu Luka", "pengarang": "Eka Kurniawan", "tahun": 2002, "stok": 3 },
  { "judul": "Pulang", "pengarang": "Tere Liye", "tahun": 2015, "stok": 2 },
  { "judul": "Sang Pemimpi", "pengarang": "Andrea Hirata", "tahun": 2006, "stok": 6 },
  { "judul": "Perahu Kertas", "pengarang": "Dee Lestari", "tahun": 2009, "stok": 0 },
  { "judul": "Gadis Kretek", "pengarang": "Ratih Kumala", "tahun": 2012, "stok": 4 }
]
```

Bandingkan dengan tabel Daftar Buku yang sudah kamu kenal sejak [dokumentasi jobsheet-01](#): 5 buku pertama **persis sama** dengan data dummy yang dulu ditulis manual di HTML — hanya sekarang formatnya JSON, dan **ditambah** 5 buku baru (total jadi 10), yang tidak akan pernah tampil kalau kamu membuka HTML lama karena baris HTML lama memang statis dan tidak terhubung ke file ini sama sekali.

3.2 data/anggota.json — 4 Objek Anggota

```
[
  { "no_anggota": "A001", "nama": "Siti Aminah", "alamat": "Malang", "no_hp": "0812xxxx" },
  { "no_anggota": "A002", "nama": "Budi Santoso", "alamat": "Batu", "no_hp": "0813xxxx" },
  { "no_anggota": "A003", "nama": "Dewi Lestari", "alamat": "Malang", "no_hp": "0814xxxx" },
]
```

```
{ "no_anggota": "A004", "nama": "Rizky Firmansyah", "alamat": "Lawang", "no_hp": "0815xxxx" }
]
```

Sama polanya: 2 anggota pertama sama dengan data dummy lama di [dokumentasi jobsheet-01](#), ditambah 2 anggota baru.

3.3 Kenapa Nama Kuncinya Persis Sama dengan Atribut `name` di Form?

Perhatikan kunci-kunci JSON ini — `judul`, `pengarang`, `tahun`, `stok` untuk buku; `no_anggota`, `nama`, `alamat`, `no_hp` untuk anggota — **persis sama** dengan atribut `name` pada input form Tambah Buku/Tambah Anggota yang sudah kamu pelajari di [dokumentasi jobsheet-01 §4](#) dan [§6](#). Ini **bukan kebetulan** — penamaan yang konsisten di seluruh aplikasi (HTML, JSON, dan nanti database sungguhan) membuat data jauh lebih mudah dilacak: satu field yang sama selalu punya **nama yang sama** di setiap lapisan, tidak perlu "menerjemahkan" nama field yang berbeda-beda di setiap tempat.

3.4 Tipe Data di Dalam JSON

JSON mendukung beberapa tipe nilai dasar, dan kedua file ini memakai 2 di antaranya:

Tipe	Contoh di Data Ini	Ciri
String (teks)	"Laskar Pelangi", "A001"	Selalu diapit tanda kutip dua.
Number (angka)	2005, 4, 0	Tidak diapit tanda kutip.

Perhatikan `"tahun": 2005` dan `"stok": 4` ditulis **tanpa** tanda kutip — artinya JavaScript akan membacanya sebagai **angka sungguhan**, bukan teks. Ini penting: kalau kamu menuliskan `"tahun": "2005"` (dengan kutip), nilainya akan jadi teks `"2005"`, yang meskipun terlihat sama di layar, **tidak bisa** langsung dipakai untuk perbandingan angka seperti `tahun > 2000` tanpa dikonversi dulu (ingat `parseInt()` yang dipakai untuk keperluan serupa di [dokumentasi jobsheet-05 §7.6](#)). Sebaliknya, perhatikan `"no_anggota": "A001"` sengaja ditulis sebagai **string** (bukan angka) — konsisten dengan pembahasan [dokumentasi jobsheet-01 §6.4](#) kenapa nomor anggota memakai huruf+angka, jadi tidak mungkin disimpan sebagai tipe angka murni.

3.5 Bagaimana Data Ini "Berubah" Jadi Objek JavaScript?

File `.json` ini **hanyalah teks** yang disimpan di server/folder — belum menjadi objek JavaScript yang bisa diakses lewat `buku.judul` sampai benar-benar **diambil dan diuraikan (parse)** oleh kode. Proses ini dilakukan oleh method `.json()` yang dipanggil setelah `fetch()` berhasil — dibahas detail langkah demi langkah di [bab 4 §4.5](#).

Lanjut ke: [JS: Mengambil & Menampilkan Daftar Buku](#)

4. JS: Mengambil & Menampilkan Daftar Buku

Ini fungsi paling penting di jobsheet ini — menggabungkan **semua** konsep dari [bab 1](#) jadi satu alur nyata.

4.1 Kode Lengkap

```
// Mengambil & menampilkan Daftar Buku secara asinkron dari data/buku.json
async function muatDaftarBuku() {
  const tbody = document.querySelector(".table-responsive table tbody");
  const loading = document.getElementById("loading-indicator");
  if (!tbody) return;

  loading.style.display = "block";
  tbody.innerHTML = "";

  try {
    // simulasi delay jaringan agar loading indicator terlihat
    await new Promise((resolve) => setTimeout(resolve, 600));

    const res = await fetch("../data/buku.json");
    if (!res.ok) {
      throw new Error("Gagal mengambil data (status " + res.status + ")");
    }
    const daftarBuku = await res.json();

    daftarBuku.forEach(function (buku) {
      const tr = document.createElement("tr");
      tr.innerHTML =
        "<td>" + buku.judul + "</td>" +
        "<td>" + buku.pengarang + "</td>" +
        "<td>" + buku.tahun + "</td>" +
        "<td>" + buku.stok + "</td>" +
        "<td>" +
```

```

        "<button type=\"button\">Edit</button> " +
        "<button type=\"button\" class=\"btn-hapus\">Hapus</button>" +
        "</td>";
        tbody.appendChild(tr);
    });
} catch (err) {
    tbody.innerHTML =
        "<tr><td colspan=\"5\">Gagal memuat data: " + err.message + "</td></tr>";
} finally {
    loading.style.display = "none";
}
}

document.addEventListener("DOMContentLoaded", muatDaftarBuku);

```

4.2 Mengambil Elemen yang Dibutuhkan

```

const tbody = document.querySelector(".table-responsive table tbody");
const loading = document.getElementById("loading-indicator");
if (!tbody) return;

```

- `tbody` — elemen `<tbody>` kosong yang sudah dibahas di [bab 2 §2.1](#), tempat baris-baris hasil `fetch` nanti akan disisipkan.
- `loading` — elemen indikator "Memuat data..." dari [bab 2 §2.2](#).
- Guard clause `if (!tbody) return;` — pola yang sama dari [dokumentasi jobsheet-05 §1.7](#). Perhatikan hanya `tbody` yang diperiksa (bukan `loading` juga) — ini sedikit tidak konsisten dengan pola guard clause yang lebih lengkap di `app.js` ([dokumentasi jobsheet-05 §4.3](#)), tapi aman **untuk saat ini** karena `buku.js` hanya dimuat di halaman yang memang selalu punya kedua elemen tersebut (ingat dari [bab 2 §2.3](#)).

4.3 Menampilkan (dan Menyembunyikan) Loading Indicator

```

loading.style.display = "block";
tbody.innerHTML = "";

```

- `loading.style.display = "block";` — mengubah gaya `display` elemen loading indicator secara langsung dari JavaScript (sama konsepnya dengan `row.style.display` yang sudah kamu pakai di [dokumentasi jobsheet-05 §6.5](#)), menimpa `style="display:none;"` bawaan dari HTML ([bab 2 §2.2](#)) — teks "Memuat data..." pun muncul.

- `tbody.innerHTML = ""`; — mengosongkan isi `<tbody>` (berjaga-jaga kalau fungsi ini dipanggil lebih dari sekali, misalnya kalau nanti ditambah tombol "Muat Ulang").

Di akhir fungsi (baik berhasil maupun gagal), baris `loading.style.display = "none"`; di dalam blok `finally` (§4.8) akan menyembunyikannya kembali.

4.4 Simulasi Delay Jaringan

```
await new Promise((resolve) => setTimeout(resolve, 600));
```

Ingat dari [dokumentasi jobsheet-06 §1.4](#), `fetch()` sendiri sudah mengembalikan Promise. Baris ini **membuat** Promise baru secara manual** yang sengaja "menunda" selama 600** milidetik (`setTimeout(resolve, 600)`) sebelum melanjutkan — murni untuk **keperluan belajar**, supaya loading indicator sempat terlihat oleh mata (karena mengambil file JSON lokal biasanya **sangat cepat**, kurang dari sepersekian detik, sehingga tanpa delay buatan ini, teks "Memuat data..." mungkin tidak sempat terlihat sama sekali). Ingat catatan dari [README.md](#) jobsheet ini: delay ini memang disimulasikan, bukan delay jaringan sungguhan.

4.5 Mengambil Data dan Memeriksa Keberhasilannya

```
const res = await fetch("../data/buku.json");
if (!res.ok) {
    throw new Error("Gagal mengambil data (status " + res.status + ")");
}
const daftarBuku = await res.json();
```

- `fetch("../data/buku.json")` — perhatikan path relatifnya `../` karena `buku.js` dimuat dari halaman `buku/list.html` (di dalam folder `buku/`), sedangkan `data/buku.json` berada di folder `data/` yang sejajar (`../` naik dulu ke root, ingat aturan path relatif dari [dokumentasi jobsheet-01 §1.5](#)).
- `res.ok` — properti pada hasil `fetch` yang bernilai `true` kalau permintaan berhasil (status HTTP 200-an), `false` kalau gagal (misalnya file tidak ditemukan, status 404). Menariknya, `fetch()` **tidak otomatis** dianggap gagal (tidak masuk ke `catch`) hanya karena file tidak ditemukan — itulah kenapa perlu pengecekan manual `if (!res.ok)`.
- `throw new Error("...")` — membuat dan "melempar" sebuah objek Error secara manual. Melempar error di sini **sengaja** memindahkan alur eksekusi langsung ke blok `catch` di bawah (§4.7), seakan-akan terjadi error sungguhan — trik umum untuk mengubah "kegagalan logis" (fetch "berhasil" secara teknis tapi filenya ternyata tidak ada) menjadi alur penanganan error yang seragam.

- `await res.json()` — mengubah isi respons (yang awalnya berupa teks JSON mentah) menjadi **array objek JavaScript** sungguhan yang bisa diakses lewat `.judul`, `.pengarang`, dst. (ingat pembahasan konsep ini di [dokumentasi jobsheet-06 §3.5](#)). Perhatikan `.json()` **juga** memakai `await` — ia sendiri adalah proses asinkron terpisah (membaca dan menguraikan teks butuh waktu juga, meski biasanya sangat singkat).

4.6 Membuat Baris Tabel dari Data

```
daftarBuku.forEach(function (buku) {
  const tr = document.createElement("tr");
  tr.innerHTML =
    "<td>" + buku.judul + "</td>" +
    "<td>" + buku.pengarang + "</td>" +
    "<td>" + buku.tahun + "</td>" +
    "<td>" + buku.stok + "</td>" +
    "<td>" +
    "<button type='button'>Edit</button> " +
    "<button type='button' class='btn-hapus'>Hapus</button>" +
    "</td>";
  tbody.appendChild(tr);
});
```

- `daftarBuku.forEach(...)` — mengulang **setiap** objek buku di dalam array hasil `res.json()` (pola `forEach` yang sama dengan yang sudah kamu pakai di [dokumentasi jobsheet-05 §5.2](#) dan [§6.5](#)).
- `document.createElement("tr")` — membuat elemen `<tr>` baru dari kode (ingat method serupa, `createElement("span")`), dari [dokumentasi jobsheet-05 §7.3](#)).
- `tr.innerHTML = "..."` — mengisi elemen `<tr>` yang baru dibuat itu dengan potongan HTML (dalam bentuk **teks**, digabung dengan operator `+`) berisi 5 sel `<td>`, termasuk tombol Edit dan Hapus. Perhatikan `buku.judul`, `buku.pengarang`, dst. — ini cara mengakses **nilai** dari sebuah kunci di dalam objek JavaScript (`buku` di sini adalah satu objek dari array `daftarBuku`, sesuai struktur yang sudah dibahas di [bab 3](#)).
- `tbody.appendChild(tr);` — method DOM untuk **menambahkan** elemen `tr` yang baru dibuat itu sebagai anak terakhir dari `tbody`. Method ini dipanggil **sekali per buku**, sehingga setelah seluruh `forEach` selesai, `<tbody>` akan berisi 10 baris `<tr>` (jumlah objek di `data/buku.json`, ingat dari [bab 3 §3.1](#)) — padahal di HTML aslinya `<tbody>` itu benar-benar kosong.

Catatan tentang keamanan: menulis `tr.innerHTML = "<td>" + buku.judul + "</td>"` dengan cara menggabung teks langsung seperti ini berpotensi menimbulkan celah keamanan (disebut **XSS**, *Cross-Site Scripting*) **kalau** data `buku.judul` berasal dari input pengguna yang tidak tepercaya (misalnya kalau ada pengguna jahat berhasil menyisipkan kode HTML/JavaScript ke dalam judul

buku). Di jobsheet ini amannya terjaga karena data `buku.json` sepenuhnya dikontrol oleh developer sendiri (bukan input pengguna) — tapi ini catatan penting untuk diingat ketika nanti data benar-benar berasal dari pengguna (mulai form yang tersambung ke server di jobsheet-jobsheet selanjutnya).

4.7 Menangkap dan Menampilkan Error

```
} catch (err) {  
    tbody.innerHTML =  
        "<tr><td colspan=\\\"5\\\">Gagal memuat data: " + err.message + "</td></tr>";  
}
```

Kalau **apa pun** di dalam blok `try` gagal — baik `fetch` itu sendiri gagal total (misalnya nama file salah ketik), maupun `throw new Error` di §4.5 sengaja dipanggil — kode di blok `catch` ini yang dijalankan. `err.message` berisi teks penjelasan error (baik pesan yang kita tulis sendiri lewat `throw new Error("...")`, maupun pesan bawaan browser kalau error-nya jenis lain). Perhatikan `colspan="5"` — atribut HTML yang belum pernah dipakai di jobsheet-jobsheet sebelumnya: membuat satu sel `<td>` **merentang lebar 5 kolom sekaligus** (jumlah kolom tabel Daftar Buku), supaya pesan error tampil sebagai satu baris penuh yang rapi, bukan menyempil hanya di kolom pertama.

4.8 Blok `finally`: Selalu Menyembunyikan Loading

```
} finally {  
    loading.style.display = "none";  
}
```

Ingat dari [dokumentasi jobsheet-06 §1.6](#), blok `finally` **selalu** dijalankan — entah data berhasil dimuat (§4.6) atau gagal (§4.7). Inilah alasan kenapa kode menyembunyikan kembali loading indicator diletakkan di sini, bukan di akhir blok `try` saja — kalau diletakkan di akhir `try`, baris itu **tidak akan pernah dijalankan** kalau terjadi error (karena error langsung melompat ke `catch`, melewati sisa kode di `try`), dan loading indicator akan **macet** tampil selamanya meskipun proses `fetch` sudah gagal dan berhenti.

4.9 Memanggil Fungsi Saat Halaman Siap

```
document.addEventListener("DOMContentLoaded", muatDaftarBuku);
```

Pola yang sama dengan `app.js` ([dokumentasi jobsheet-05 §1.7](#)): `muatDaftarBuku` baru dipanggil **setelah** seluruh HTML selesai dimuat menjadi DOM, memastikan elemen `<tbody>` dan `#loading-indicator` sudah pasti ada saat fungsi ini mulai mencarinya.

Lanjut ke: [JS: Mengambil & Menampilkan Daftar Anggota](#)

5. JS: Mengambil & Menampilkan Daftar Anggota

Kabar baik: `anggota.js` punya **struktur yang identik** dengan `buku.js` yang sudah kamu bedah tuntas di [bab 4](#). Bab ini hanya menyoroti bagian yang **berbeda**.

5.1 Kode Lengkap

```
// Mengambil & menampilkan Daftar Anggota secara asinkron dari data/anggota.json
async function muatDaftarAnggota() {
  const tbody = document.querySelector(".table-responsive table tbody");
  const loading = document.getElementById("loading-indicator");
  if (!tbody) return;

  loading.style.display = "block";
  tbody.innerHTML = "";

  try {
    await new Promise((resolve) => setTimeout(resolve, 600));

    const res = await fetch("../data/anggota.json");
    if (!res.ok) {
      throw new Error("Gagal mengambil data (status " + res.status + ")");
    }
    const daftarAnggota = await res.json();

    daftarAnggota.forEach(function (anggota) {
      const tr = document.createElement("tr");
      tr.innerHTML =
        "<td>" + anggota.no_anggota + "</td>" +
        "<td>" + anggota.nama + "</td>" +
        "<td>" + anggota.alamat + "</td>" +
        "<td>" + anggota.no_hp + "</td>" +
        "<td>" +
```

```

        "<button type=\"button\">Edit</button> " +
        "<button type=\"button\" class=\"btn-hapus\">Hapus</button>" +
        "</td>";
        tbody.appendChild(tr);
    });
} catch (err) {
    tbody.innerHTML =
        "<tr><td colspan=\"5\">Gagal memuat data: " + err.message + "</td></tr>";
} finally {
    loading.style.display = "none";
}
}

document.addEventListener("DOMContentLoaded", muatDaftarAnggota);

```

5.2 Perbandingan Langsung dengan buku.js

Bagian	buku.js	anggota.js
Nama fungsi	muatDaftarBuku	muatDaftarAnggota
Sumber data	fetch("../data/buku.json")	fetch("../data/anggota.json")
Nama variabel array hasil	daftarBuku	daftarAnggota
Nama variabel per-item di <code>forEach</code>	buku	anggota
Kunci objek yang diakses	.judul, .pengarang, .tahun, .stok	.no_anggota, .nama, .alamat, .no_hp
Event pemicu	DOMContentLoaded → muatDaftarBuku	DOMContentLoaded → muatDaftarAnggota

Semua bagian lain — pengambilan elemen ([dokumentasi bab 4 §4.2](#)), loading indicator ([§4.3](#)), delay simulasi ([§4.4](#)), pengecekan `res.ok` ([§4.5](#)), penanganan error ([§4.7](#)), dan blok `finally` ([§4.8](#)) — **identik persis**, cukup baca ulang [bab 4](#) kalau ada bagian yang lupa.

5.3 Kenapa Kolom yang Diakses Berbeda?

Perbedaan paling penting untuk diperhatikan ada di bagian `forEach: anggota.no_anggota, anggota.nama, anggota.alamat, anggota.no_hp` — empat kunci ini **harus** sesuai persis dengan nama kunci yang sungguhan ada di `data/anggota.json` (ingat strukturnya dari [bab 3 §3.2](#)). Kalau salah ketik nama kunci (misalnya menulis `anggota.nomor` padahal di JSON kuncinya `no_anggota`), JavaScript **tidak akan error** — ia hanya akan mengembalikan `undefined` (nilai "tidak ada"), sehingga sel tabel yang seharusnya berisi data malah tampil kosong atau bertuliskan "undefined". Ini kesalahan umum yang mudah terlewat karena tidak memunculkan error yang jelas di Console — kalau tabel tampil tapi datanya kosong/aneh, periksa dulu apakah nama kunci di kode sudah sesuai persis dengan nama kunci di file JSON-nya.

5.4 Kenapa Ditulis di Dua File Terpisah, Bukan Satu Fungsi Generik?

Kamu mungkin bertanya: kenapa tidak dibuat **satu** fungsi generik saja yang menerima nama file JSON dan daftar kolom sebagai parameter, dipakai ulang untuk buku maupun anggota — mengurangi duplikasi kode? Ini pertanyaan yang bagus, dan jawabannya berkaitan dengan tahap belajar saat ini: menulis dua fungsi terpisah yang **mirip tapi eksplisit** lebih mudah dibaca dan ditelusuri untuk pemula, sebelum mempelajari teknik menulis fungsi yang lebih generik/reusable. Ini juga ide latihan lanjutan yang bagus untuk dicoba sendiri — lihat [bab 8](#).

Lanjut ke: [JS: Event Delegation pada Tombol Hapus](#)

6. JS: Event Delegation pada Tombol Hapus

Perubahan kecil di `app.js` ini punya alasan besar di baliknya — dan memperkenalkan pola penting bernama **event delegation**.

6.1 Kode Sebelum dan Sesudah

Sebelumnya (jobsheet-05):

```
function initHapusConfirm() {
  document.querySelectorAll(".btn-hapus").forEach(function (btn) {
    btn.addEventListener("click", function () {
      const row = btn.closest("tr");
      const nama = row ? row.querySelector("td")?.textContent : "data ini";
      const yakin = confirm("Yakin ingin menghapus \"" + nama + "\"?");
```

```

        if (yakin && row) {
            row.remove();
        }
    });
});
}

```

Sekarang (jobsheet-06):

```

// Memakai event delegation di document karena baris tabel sekarang
// dirender dinamis via fetch (lihat buku.js/anggota.js) sehingga
// tombol .btn-hapus belum tentu ada saat DOMContentLoaded.
function initHapusConfirm() {
    document.addEventListener("click", function (e) {
        const btn = e.target.closest(".btn-hapus");
        if (!btn) return;

        const row = btn.closest("tr");
        const nama = row ? row.querySelector("td")?.textContent : "data ini";
        const yakin = confirm("Yakin ingin menghapus \"" + nama + "\"?");
        if (yakin && row) {
            row.remove();
        }
    });
}

```

6.2 Apa Masalah yang Diperbaiki di Sini?

Ingat cara kerja `initHapusConfirm` versi jobsheet-05 ([dokumentasi jobsheet-05 §5.2](#)): `document.querySelectorAll(".btn-hapus")` mencari **semua** tombol Hapus yang **sudah ada di DOM saat itu juga**, lalu memasang event listener ke masing-masing satu per satu. Kode ini dijalankan sekali saat `DOMContentLoaded` ([dokumentasi jobsheet-05 §1.7](#)).

Masalahnya: di jobsheet-06, tombol `.btn-hapus` **tidak lagi ada** di HTML sejak awal — ingat dari [bab 2 §2.1](#), `<tbody>` sekarang **kosong**, dan baris-baris tabel (termasuk tombol Hapus di dalamnya) baru **dibuat belakangan** oleh `buku.js / anggota.js` lewat `tbody.appendChild(tr)` (ingat dari [bab 4 §4.6](#)) — dan proses itu sendiri baru selesai **setelah** proses `fetch` asinkron rampung (butuh waktu, apalagi dengan delay simulasi 600ms dari [bab 4 §4.4](#)).

Kalau `initHapusConfirm` versi lama tetap dipakai: saat `DOMContentLoaded` terpicu dan `querySelectorAll(".btn-hapus")` dijalankan, **belum ada** satu pun** tombol `.btn-hapus` di DOM (karena `buku.js` belum selesai** mengambil data) — akibatnya, **tidak ada event listener yang**

terpasang sama sekali**, dan tombol Hapus yang muncul belakangan tidak akan** pernah bereaksi saat diklik.

6.3 Solusi: Event Delegation

Event delegation adalah teknik memasang **satu** event listener di elemen **leluhur** (di sini: `document`, elemen paling atas/luar dari semuanya) alih-alih memasang listener terpisah ke setiap elemen target. Teknik ini memanfaatkan sifat alami event di browser yang disebut **event bubbling** — ketika sebuah elemen diklik, "kejadian" klik itu **menjalar ke atas** melalui seluruh elemen leluhurnya (dari tombol → `<td>` → `<tr>` → `<tbody>` → ... → `document`), bukan cuma terjadi di elemen yang diklik itu sendiri.

```
document.addEventListener("click", function (e) {
  const btn = e.target.closest(".btn-hapus");
  if (!btn) return;
  // ...
});
```

- Event listener dipasang **satu kali saja**, di `document` — elemen yang **selalu ada** sejak awal, tidak peduli baris tabel mana pun yang belum/sudah dibuat.
- `e.target` — properti pada objek event (`e`) yang merujuk ke elemen **paling spesifik** yang benar-benar diklik pengguna (bisa jadi tombolnya sendiri, atau kadang elemen lain di dalamnya).
- `e.target.closest(".btn-hapus")` — ingat method `.closest()` dari [dokumentasi jobsheet-05 §5.3](#), tapi kali ini dipakai untuk tujuan berbeda: mencari **ke atas** dari elemen yang diklik, untuk memastikan klik itu **benar-benar terjadi** pada (atau di dalam) sebuah tombol `.btn-hapus` — karena `document` menerima event klik dari **mana saja** di seluruh halaman (termasuk klik di tombol Edit, klik di kolom pencarian, dll).
- `if (!btn) return;` — kalau klik ternyata terjadi di tempat lain (bukan tombol Hapus), `.closest(".btn-hapus")` akan mengembalikan `null`, dan fungsi berhenti di sini — tidak melakukan apa-apa untuk klik yang tidak relevan.

Setelah `btn` ditemukan, sisa kodenya (mencari `row` lewat `btn.closest("tr")`, menampilkan `confirm()`, memanggil `row.remove()`) **persis sama** dengan versi jobsheet-05 — konsep-konsep itu sudah dibahas tuntas di [dokumentasi jobsheet-05 §5.3-5.6](#).

6.4 Kenapa Ini Berhasil untuk Tombol yang "Belum Ada"?

Ini bagian paling penting untuk dipahami: karena event listener dipasang di `document` (bukan di tombolnya masing-masing), listener ini **tidak peduli** kapan tombol `.btn-hapus` itu dibuat — entah sudah ada sejak `DOMContentLoaded`, atau baru muncul beberapa ratus milidetik kemudian setelah `fetch` selesai ([bab 4](#)). Selama tombol itu **ada di DOM saat diklik** (kapan pun itu), klik pada tombol tersebut akan tetap "menjalar" naik sampai ke `document`, dan listener yang sudah terpasang sejak awal akan tetap mendeteksinya lewat `e.target.closest(".btn-hapus")`.

6.5 Kapan Sebaiknya Memakai Event Delegation?

Situasi	Pendekatan yang Cocok
Elemen target sudah ada sejak halaman dimuat, jumlahnya tetap	Pasang listener langsung ke tiap elemen (<code>querySelectorAll(...).forEach(...)</code> , seperti dokumentasi jobsheet-05)
Elemen target dibuat/dihapus secara dinamis setelah halaman dimuat (lewat <code>fetch</code> , atau interaksi pengguna lain)	Event delegation di elemen leluhur yang stabil (seperti di jobsheet ini)

Aturan praktisnya: begitu kamu mulai membuat elemen secara dinamis lewat JavaScript (seperti `buku.js` / `anggota.js` di jobsheet ini), pertanyaan "apakah elemen ini butuh event listener, dan kapan elemen ini akan ada?" jadi penting untuk dipikirkan — kalau jawabannya "elemen ini bisa muncul kapan saja, bahkan setelah kode ini pertama kali berjalan", event delegation biasanya jadi pilihan yang lebih aman.

Lanjut ke: [Menjalankan Lewat Server Lokal \(CORS\)](#)

7. Menjalankan Lewat Server Lokal (CORS)

Ini bagian **paling penting untuk diketahui sebelum mencoba** jobsheet ini sendiri — beda dari semua jobsheet sebelumnya yang bisa langsung dibuka dengan klik dua kali file HTML-nya.

7.1 Peringatan dari README

Penting: `fetch()` ke file lokal akan diblokir kebijakan CORS jika dibuka langsung dengan `file://`. Jalankan lewat server lokal...

Kalau kamu membuka `buku/list.html` langsung dengan cara yang sama seperti jobsheet-01 sampai jobsheet-05 (klik dua kali di File Explorer), tabelnya **tidak akan pernah terisi** — akan macet menampilkan "Memuat data..." selamanya, atau langsung menampilkan pesan error "Gagal memuat data". Ini **bukan bug** di kodenya, melainkan pembatasan keamanan browser yang disebut **CORS**.

7.2 Apa itu CORS?

CORS (*Cross-Origin Resource Sharing*) adalah kebijakan keamanan browser yang membatasi permintaan `fetch()` (dan teknik AJAX serupa) antar **origin** (asal) yang berbeda. Ketika kamu membuka file HTML langsung dari File Explorer, alamatnya di browser akan terlihat seperti:

```
file:///D:/1.MateriKuliah/PemogramanWeb-2026/kode-praktikum/jobsheet-06/buku/list.html
```

Protokol `file://` ini diperlakukan browser secara **sangat ketat** — banyak browser modern (terutama Chrome) sama sekali **tidak mengizinkan** `fetch()` mengambil file lain lewat protokol ini, sebagai bagian dari kebijakan keamanannya, meskipun secara teknis file yang diminta (`data/buku.json`) berada tepat di folder yang bersebelahan di komputer yang sama. Ini beda dengan tag `<link>` atau `<script src="...">` yang sudah kamu pakai sejak jobsheet-02 dan jobsheet-05 — tag-tag itu **tidak** terkena pembatasan CORS seketat `fetch()`.

7.3 Solusi: Jalankan Lewat Server Lokal

Solusinya adalah membuka halaman ini bukan lewat `file://`, melainkan lewat `http://` — dengan menjalankan **server lokal** sederhana di komputermu sendiri. README menyarankan 2 cara:

Cara 1 — Lewat PHP (kalau PHP sudah terpasang di komputermu):

```
php -S localhost:8000
```

Jalankan perintah ini di folder `jobsheet-06/` lewat terminal, lalu buka `http://localhost:8000/index.html` di browser.

Cara 2 — Ekstensi "Live Server" di VS Code: Kalau kamu memakai editor VS Code, pasang ekstensi bernama **Live Server**, lalu klik kanan file `index.html` → "Open with Live Server". Ini

akan otomatis menjalankan server lokal dan membuka browser untukmu, tanpa perlu mengetik perintah apa pun di terminal.

Cara 3 — Laragon (Apache): kalau kamu memakai Laragon, cukup taruh folder proyek di dalam `C:\laragon\www\`, nyalakan Apache lewat menu Laragon, lalu buka `http://<nama-domain>.test/.../jobsheet-06/index.html` di browser (nama domainnya otomatis dibuat Laragon dari nama folder proyekmu). Semua path `fetch()` di jobsheet ini memakai path relatif biasa (`../data/buku.json`, dst.), jadi tetap benar diakses dari kedalaman folder mana pun.

Ketiga cara ini punya tujuan yang sama: membuat halaman diakses lewat alamat `http://...` (bukan `file://...`), sehingga `fetch()` diizinkan mengambil file JSON di folder yang sama.

7.4 Kenapa Ini Baru Muncul Sekarang, Bukan di Jobsheet Sebelumnya?

Jobsheet-01 sampai jobsheet-05 semuanya bisa dibuka langsung dengan `file://` karena **tidak ada satu pun** yang memakai `fetch()` untuk mengambil file lain — CSS (`<link>`) dan JavaScript (`<script src>`) tidak terkena pembatasan CORS seketat ini. Baru di jobsheet-06, dengan diperkenalkannya `fetch()` untuk mengambil `data/buku.json` dan `data/anggota.json`, batasan CORS ini pertama kali "terasa" — dan memang sengaja diperkenalkan bersamaan dengan `fetch()` di jobsheet ini karena keduanya **selalu berkaitan** dalam praktik pengembangan web sungguhan.

7.5 Cara Menguji Penanganan Error

Sesuai catatan di [README.md](#) jobsheet ini:

Uji error handling dengan mengganti sementara nama file di `fetch(...)` menjadi nama yang salah.

Coba lakukan ini sebagai latihan: buka `assets/js/buku.js`, ubah baris `fetch("../data/buku.json")` menjadi misalnya `fetch("../data/bukuu.json")` (sengaja salah ketik), simpan, lalu buka halaman `buku/list.html` lewat server lokal. Kamu akan melihat pesan error "Gagal memuat data: Gagal mengambil data (status 404)" tampil di dalam tabel — ini membuktikan langsung bahwa blok `try / catch` yang sudah dibahas di [bab 4 §4.7](#) benar-benar bekerja, bukan cuma teori. Setelah selesai mencoba, jangan lupa kembalikan nama filenya ke `buku.json` semula.

Lanjut ke: [Rangkuman & Latihan Lanjutan](#)

8. Rangkuman & Latihan Lanjutan

8.1 Rangkuman Keseluruhan Jobsheet 6

Bagian	Konsep yang Dipelajari
Konsep Dasar	AJAX, format JSON, <code>fetch()</code> , Promise, <code>async / await</code> , <code>try / catch / finally</code>
Perubahan HTML	<code><tbody></code> kosong, loading indicator, urutan <code><script></code>
Data JSON	Struktur <code>buku.json / anggota.json</code> , tipe data JSON
Fetch & Render Buku	Alur lengkap <code>muatDaftarBuku</code> : <code>fetch</code> → cek status → parse → render → error handling
Fetch & Render Anggota	Pola yang sama diterapkan ke data berbeda
Event Delegation	Kenapa & bagaimana menangani elemen yang dibuat dinamis
Server Lokal & CORS	Kenapa <code>fetch()</code> butuh <code>http://</code> , cara menjalankan server lokal

8.2 Konsep Inti yang Perlu Diingat

- `fetch()` **selalu asinkron** — hasilnya berupa Promise, butuh `await` (di dalam `async function`) untuk mendapatkan data sungguhan ([bab 1](#)).
- `res.ok` **harus diperiksa manual** — `fetch()` tidak otomatis dianggap gagal hanya karena file/endpoint tidak ditemukan ([bab 4 §4.5](#)).
- `try / catch / finally` **menjaga aplikasi tetap "hidup"** meski terjadi kegagalan jaringan — pengguna melihat pesan error yang jelas, bukan halaman rusak/kosong tanpa penjelasan ([bab 1 §1.6](#)).
- Event delegation diperlukan untuk elemen dinamis** — begitu elemen dibuat setelah `DOMContentLoaded` (lewat `fetch` atau interaksi lain), pasang listener di elemen leluhur yang stabil, bukan di elemen itu sendiri ([bab 6](#)).
- `fetch()` **butuh server, bukan** `file://` — ingat menjalankan lewat `php -S localhost:8000`, Live Server, atau Laragon setiap kali mencoba jobsheet ini ([bab 7](#)).

8.3 Cara Mencoba Sendiri

1. **Jalankan server lokal dulu** (ingat [bab 7](#) — ini **wajib**, tidak seperti jobsheet-jobsheet sebelumnya).
2. Buka `http://localhost:8000/buku/list.html` (sesuaikan port kalau berbeda). Amati sekilas teks "Memuat data..." muncul sebelum tabel terisi 10 baris buku.
3. Buka **DevTools** → **tab Network**, refresh halaman, cari permintaan ke `buku.json` — klik untuk melihat respons JSON mentahnya, bandingkan dengan isi file `data/buku.json` yang sudah kamu baca di [bab 3](#).
4. Coba fitur pencarian dan tombol Hapus seperti di jobsheet-05 — amati keduanya tetap berfungsi normal meski barisnya sekarang dibuat dinamis.
5. Praktikkan langkah menguji error di [bab 7 §7.5](#) — sengaja salah ketik nama file JSON, amati pesan error tampil rapi di dalam tabel.

8.4 Ide Latihan Tambahan (Opsional)

1. **Tambah tombol "Muat Ulang"** di halaman Daftar Buku yang, saat diklik, memanggil ulang `muatDaftarBuku()` — perhatikan fungsi ini sudah mengosongkan `tbody` terlebih dulu ([bab 4 §4.3](#)), jadi aman dipanggil berkali-kali.
2. **Satukan** `buku.js` **dan** `anggota.js` jadi satu fungsi generik yang menerima nama file JSON dan daftar nama kunci sebagai parameter — latihan langsung untuk pertanyaan yang diajukan di [bab 5 §5.4](#).
3. **Tambah kolom baru** di `data/buku.json` (misalnya "kategori"), lalu tampilkan di tabel dengan menambah satu `<th>` di HTML dan satu `<td>` di `tr.innerHTML` pada `buku.js`.
4. **Uji delegasi event lebih jauh** — tambahkan `console.log(e.target)` di awal fungsi `initHapusConfirm` ([bab 6](#)), buka Console, lalu klik berbagai tempat di halaman (bukan cuma tombol Hapus) untuk melihat sendiri bagaimana `document` menerima **semua** event klik di halaman, dan bagaimana `e.target.closest(...)` menyaring hanya yang relevan.
5. **Ganti delay simulasi** di [bab 4 §4.4](#) dari 600 menjadi 3000 (3 detik), amati loading indicator jadi jauh lebih terlihat — ini juga cara yang baik untuk merasakan pentingnya loading indicator pada koneksi yang lambat.

Kalau ada bagian yang masih membingungkan, coba baca ulang [bab 1](#) sambil membuka DevTools Console — ketik `await fetch("../data/buku.json").then(r => r.json())` langsung di Console (dari halaman `buku/list.html` yang dibuka lewat server lokal) untuk melihat sendiri data yang dikembalikan, tanpa perlu membaca lewat tabel.